

speed-test

July 16, 2026

1 Speed Testing OSKAR

Note all units of time are measured in seconds. Every other measurement is a simple amount e.g. number of sources or number of blocks.

1.1 Initial Imports

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
```

1.2 Converting raw speeds to detailed info

```
[2]: # Load CSV
raw = pd.read_csv("speed_raw.csv")

raw
```

```
[2]:      x    y    z      dt  db    b
0  400  400  400  62751.015  21  150
1   10   10  100    58.824  15   39
2  400  400    1   744.814   2    3
3  100  100  100  2084.739   6   39
```

```
[3]: # Calculate box side length, channel area, area side length, and total source_
↳count
raw.insert(3, "n", raw.x * raw.y * raw.z)
raw.insert(3, "a", raw.x * raw.y)
raw.insert(3, "l", np.sqrt(raw.a))
raw.insert(3, "k", np.cbrt(raw.n))

# Calculate total elapsed time and time per block
raw.insert(9, "dt/db", raw.dt / raw.db)
raw.insert(11, "t", raw["dt/db"] * raw.b)

raw
```

```
[3]:
```

	x	y	z	k	l	a	n	dt	db	\
0	400	400	400	400.000000	400.0	160000	64000000	62751.015	21	
1	10	10	100	21.544347	10.0	100	10000	58.824	15	
2	400	400	1	54.288352	400.0	160000	160000	744.814	2	
3	100	100	100	100.000000	100.0	10000	1000000	2084.739	6	

	dt/db	b	t
0	2988.143571	150	448221.535714
1	3.921600	39	152.942400
2	372.407000	3	1117.221000
3	347.456500	39	13550.803500

```
[4]: # Save data to the calculation CSV
raw.to_csv("speed_all.csv", index=False)
```

1.3 Calculate power law

The tight-bound time complexity should be dependent on three variables:

$$t_{\max} = \Theta(g(x, y, z)); g(x, y, z) = \alpha(xy)^{\beta}z^{\gamma} : \alpha, \beta, \gamma \in \mathbb{R}^{+}$$

Where α, β, γ are to be calculated using a formula fit. Since we expect $x = y$, we can simplify this to the following formula:

$$g(x, y, z) \equiv g(a, z) = \alpha a^{\beta} z^{\gamma}, a = x \cdot y$$

To make the fitting easier, we can plot the data on a log-log scale and use MLR to more easily evaluate the data:

$$\log(g(a, z)) = f(a, z) = \log(\alpha) + \beta \log(a) + \gamma \log(z) \equiv F(A, Z) = \delta + \beta A + \gamma Z : \delta = \log(\alpha)$$

```
[5]: # Load from the CSV
data = pd.read_csv("speed_all.csv")
```

```
[6]: # Get column data and convert to the log form
i = data[['a', 'z']]
I = np.log(data[['a', 'z']])
F = pd.Series(np.log(data.t))

print("x-axis values:")
print(I)
print("\ny-axis values:")
print(F)
```

x-axis values:

	a	z
0	11.982929	5.991465
1	4.605170	4.605170
2	11.982929	0.000000
3	9.210340	4.605170

```
y-axis values:
0    13.013043
1     5.030061
2     7.018600
3     9.514201
Name: t, dtype: float64
```

```
[7]: # Create the MLR model
model = LinearRegression()

model.fit(I, F)
```

```
[7]: LinearRegression()
```

```
[8]: # Set variables corresponding to parameter values

delta = model.intercept_
[beta, gamma] = model.coef_
alpha = np.exp(delta)

print(alpha, beta, gamma)
```

```
0.02553200856918349 0.8949803733897755 1.0152485906089168
```

```
[9]: # Test the parameters on the same data

test_F = model.predict(I)
test_g = np.exp(test_F)

cdata = pd.DataFrame({"real" : data.t, "pred" : test_g})

print(cdata)
```

	real	pred
0	448221.535714	508637.019415
1	152.942400	168.866270
2	1117.221000	1160.567025
3	13550.803500	10411.268728

```
[10]: # Check relative error
print((cdata.real-cdata.pred)/cdata.real)

avg_error = np.mean(np.abs((cdata.real-cdata.pred)/cdata.real))

print(avg_error)
```

```
0    -0.134789
1    -0.104117
2    -0.038798
```

```
3      0.231686
dtype: float64
0.12734760825875616
```

```
[ ]: # Final formula
print("The final time complexity formula is:")
print("g(x,y,z) = %(alpha)5.4f * (x * y) ** %(beta)5.4f * z ** %(gamma)5.4f" %
      ↪{ 'alpha':alpha, 'beta':beta, 'gamma':gamma })
```

The final time complexity formula is:

$g(x,y,z) = 0.0255 * (x * y) ** 0.8950 * z ** 1.0152$

Or in $\text{\LaTeX}$ we have:

$$\Theta\left(0.0255(xy)^{0.8950}z^{1.0152}\right) \approx \Theta(k^{2.8052}) \equiv \Theta(n^{0.9351})$$

Where k is the average side length of a simulation box (with time complexity being exact), and n is the number of sources in a box (with time complexity being averaged).

1.4 Computer specifications

1.4.1 General Information

Info	Data
Model	NVIDIA GeForce RTX 2070
Clock	33 MHz
VRAM	8 GiB
Max Power	175 W
Compute Cap.	7.5

1.4.2 While OSKAR is Running in the Background

Assuming a nice value of 0.

Info	Data
Performance State	P5
VRAM Usage	96 MiB

(that's all I could collect in a reasonable amount of time)